2

Создание хранимых функций

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- Использование функций
- Создание хранимых функций
- Вызов функции
- Удаление функции
- Разница между процедурой и функцией

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цель урока

На этом уроке вы научитесь создавать и выполнять функции.

Обзор хранимых функций

- **Функция – именованный блок PL/SQL, возвращающий значение.**
- **Может храниться как объект схемы в базе данных для повторного выполнения.**
- **Функция используется в выражении или вызывается для предоставления значения параметра.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор хранимых функций

Функция – это именованный блок PL/SQL, который может вызываться, принимать параметры и возвращать значение. Вообще говоря, функции используются для вычисления значений. Структурно функции и процедуры похожи. Однако функция должна возвращать значение в вызывающую среду, а процедура может совсем не возвращать или возвращает несколько значений. Как и процедура, функция имеет заголовок, декларативную секцию, исполняемую секцию и необязательную секцию обработки исключений. Функция должна содержать предложение `RETURN` в заголовке и, по крайней мере, один оператор `RETURN` в исполняемой секции.

Функции (*functions*) могут храниться в базе данных как объекты схемы для повторного выполнения. Функция, хранимая в базе данных, называется хранимой функцией. Функции могут также создаваться в приложении на стороне клиента.

Они могут использоваться повторно в любом количестве приложений. При изменении требований к обработке необходимо изменить только функцию, что значительно упрощает сопровождение.

Функции могут вызываться как часть SQL-выражения или как часть PL/SQL-выражения. В SQL-выражении функция должна подчиняться определенным правилам во избежании побочных эффектов. В PL/SQL-выражении идентификатор функции действует как переменная, значение которой зависит от передаваемых параметров.

Создание функции: синтаксис

Блок PL/SQL должен содержать, по крайней мере, один оператор RETURN.

```
CREATE [OR REPLACE] FUNCTION имя_функции
 [(параметр1 [режим1] тип_данных1, ...)]
RETURN тип_данных IS|AS
 [объявления_локальных_переменных; …]
BEGIN
  -- действия;
  RETURN выражение;
END [имя_функции];
```

Блок PL/SQL

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание функции: синтаксис

Функция – это блок PL/SQL, возвращающий значение. В нем должен присутствовать оператор RETURN, который возвращает значение с таким типом данных, который был указан в объявлении функции.

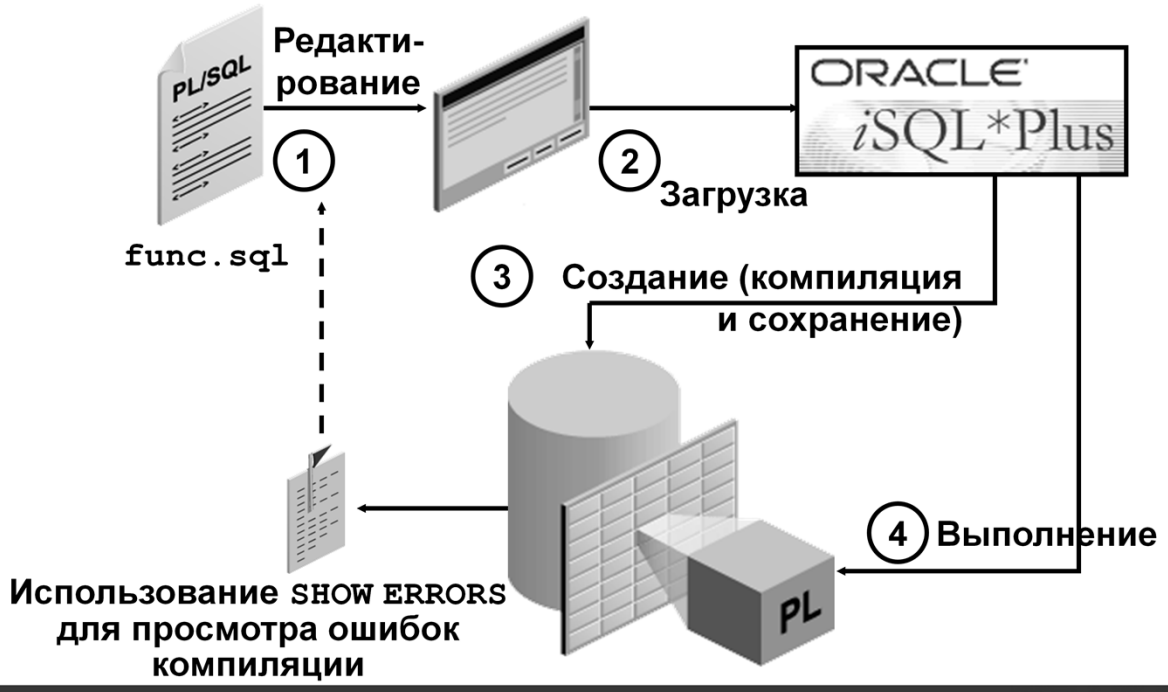
Новые функции создаются с помощью команды CREATE FUNCTION. Функция может содержать список параметров, должна возвращать одно значение и определять действия, выполняемые стандартным блоком PL/SQL.

В команде CREATE FUNCTION:

- Параметр REPLACE означает, что при наличии функции с таким же именем она удаляется и заменяется новым вариантом, созданным данной командой.
- *Тип данных в предложении* RETURN (RETURN *data type*) не должен включать указание размера.
- Блок PL/SQL начинается с оператора BEGIN или объявления локальных переменных, а заканчивается оператором END или END *имя_функции*.
- В блоке PL/SQL должен присутствовать, по крайней мере, один оператор RETURN *выражение*.
- В PL/SQL-блоке хранимой функции нельзя ссылаться на хост- или связанные переменные.

Примечание. Хотя режимы OUT и IN OUT допускаются, их использование в функциях является плохой практикой программирования..Когда требуется вернуть из функции несколько значений, используйте составной тип в качестве типа возвращаемого значения (например, запись или таблицу PL/SQL).

Разработка функций



Copyright © 2004, Oracle. All rights reserved.

Как разрабатываются хранимые функции

Диаграмма иллюстрирует основные шаги по разработке хранимой функции. Чтобы создать хранимую функцию выполните следующие шаги:

1. Напишите код для создания функции в редакторе или текстовом процессоре, а затем сохраните его в скрипт-файле SQL (обычно имеющем расширение `.sql`).
2. Загрузите код функции из файла в буфер, используя *i*SQL*Plus в качестве среды разработки.
3. Выполните команды `CREATE FUNCTION` для компилирования и сохранения функции в базе данных.
4. После успешной компиляции вызовите функцию из среды PL/SQL или приложения.

Возвращаемое значение

- Включите фразу `RETURN` с указанием типа данных в заголовок функции.
- Включите, по крайней мере, один *оператор* `RETURN` (`RETURN` *statement*) в исполняемую секцию.

Несмотря на то что можно указать несколько операторов `RETURN` в теле функции (обычно внутри оператора `IF`), выполняется только один оператор `RETURN`, так как при этом после возврата значения выполнение блока прекращается.

Для просмотра ошибок компиляции используйте команды *i*SQL*Plus `SHOW ERRORS` или `SHOW ERRORS FUNCTION имя_функции`.

Пример хранимой функции

- **Создание функции:**

```
CREATE OR REPLACE FUNCTION get_sal
 (id employees.employee_id%TYPE) RETURN NUMBER IS
  sal employees.salary%TYPE := 0;
BEGIN
  SELECT salary
  INTO   sal
  FROM   employees
  WHERE  employee_id = id;
  RETURN sal;
END get_sal;
/
```

- **Вызов функции из выражения или при передаче значения параметра:**

```
EXECUTE dbms_output.put_line(get_sal(100))
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Пример хранимой функции

Создается функция `get_sal` с одним входным параметром, возвращающая числовое значение оклада. Чтобы создать функцию, выполните приведенную на слайде команду или сохраните ее в скрипт-файле, который затем выполните.

Хорошей практикой программирования считается использование единственного оператора `RETURN`, возвращающего значение, присвоенное локальной переменной. Когда функция содержит секцию обработки исключений, в ней также может располагаться оператор `RETURN`.

Так как функция возвращает значение, ее можно вызывать из выражении PL/SQL.

Во втором примере, приведенном на слайде, для вызова процедуры `DBMS_OUTPUT.PUT_LINE` используется команда *i*SQL*Plus `EXECUTE`. Аргумент вызываемой процедуры представляет из себя значение, возвращаемое функцией `get_sal`. В этом случае сначала вызывается функция `get_sal`, чтобы получить оклад сотрудника с номером 100. Возвращаемая величина оклада передается в качестве параметра процедуре `DBMS_OUTPUT.PUT_LINE`, которая выводит результат (при условии, что была выполнена команда `SET SERVEROUTPUT ON`).

Примечание. Функция должна возвращать значение. В приведенном примере значение не возвращается, когда не находится строка для переданного номера сотрудника (`id`). В общем случае в коде должен быть обработчик исключения, чтобы в такой ситуации также возвращалось значение.

Способы вызова функций

- **Вызов из выражения PL/SQL**
  - **Использование хост-переменной для получения результата**

```
VARIABLE salary NUMBER
EXECUTE :salary := get_sal(100)
```

  - **Использование локальной переменной для получения результата**

```
DECLARE sal employees.salary%type;
BEGIN
  sal := get_sal(100); ...
END;
```

- **Вызов при передаче параметра другой подпрограмме**

```
EXECUTE dbms_output.put_line(get_sal(100))
```

- **В команде SQL (с некоторыми ограничениями)**

```
SELECT job_id, get_sal(employee_id) FROM employees;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Способы вызова функций

Функции с мощными возможностями создаются в результате хорошо продуманного проектирования. Они могут быть вызваны следующими способами.

- **Как часть выражений PL/SQL.** Для хранения значения, возвращаемого функцией, можно использовать хост-переменную или локальную переменную. В первом примере, приводимом на слайде, используется хост-переменная, во втором – локальная переменная анонимного блока.
- **Как параметр другой подпрограммы.** Это демонстрируется в третьем примере. Функция `get_sal` вместе с ее аргументом указана в виде параметра в процедуре `DBMS_OUTPUT.PUT_LINE`. Использование *вложенных функций* (*nesting functions*) рассматривается в курсе *Введение в Oracle 10g: основы SQL, часть I.*
- **Как выражение в команде.** В последнем примере показывается, как можно использовать однострочную функцию в команде SQL.

Примечание: преимущества и ограничения применения функций в командах SQL рассматриваются на следующих страницах.

Преимущества применения пользовательских функций в командах SQL

- **Расширяют возможности SQL, когда обработка слишком сложна, неудобна или не может быть выполнена стандартными средствами SQL.**
- **С помощью пользовательских функций в предложении `WHERE` можно эффективно производить фильтрацию данных и не делать этого в коде приложения.**
- **Манипулирование данными можно осуществлять путем применения пользовательских функций.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Преимущества применения пользовательских функций в командах SQL

Команды SQL могут содержать ссылки на пользовательские функции, написанные на PL/SQL. Например, пользовательская функция может находиться в любом месте, где допустима встроенная функция SQL (такая как `UPPER`).

Преимущества

- Возможность производить вычисления, слишком сложные, неудобные или вообще невозможные в SQL.
- Увеличение независимости данных за счет того, что сложный анализ данных производится на сервере Oracle вместо того, чтобы считывать эти данные в приложение.
- Увеличение эффективности запросов за счет того, что функция выполняется не в приложении, а в запросе.
- Возможность манипулирования новыми типами данных (например, широтой и долготой) благодаря тому, что символьные строки кодируются, а затем для работы с ними используются функции.

Пример вызова функций в SQL-выражениях

```
CREATE OR REPLACE FUNCTION tax(value IN NUMBER)
 RETURN NUMBER IS
BEGIN
   RETURN (value * 0.08);
END tax;
/
SELECT employee_id, last_name, salary, tax(salary)
FROM   employees
WHERE  department_id = 100;
```

```
Function created.
```

EMPLOYEE_ID	LAST_NAME	SALARY	TAX(SALARY)
108	Greenberg	12000	960
109	Faviet	9000	720
110	Chen	8200	656
111	Sciarra	7700	616
112	Urman	7800	624
113	Popp	6900	552

```
6 rows selected.
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Пример вызова функций в SQL-выражениях

На слайде показано создание функции `tax`, предназначенной для подсчета подоходного налога. Функции передается параметр типа `NUMBER`, а она возвращает значение этого параметра, умноженное на 0.08, в качестве рассчитанной величины налога.

В *i*SQL*Plus функция `tax` вызывается (как выражение) из предложения `SELECT`, в котором задается также вывод номера служащего, его фамилии, оклада и налога. В результате приводимого на слайде запроса выводятся данные для сотрудников отдела `100`.

Вызов пользовательских функций

Пользовательские функции выполняются как встроенные однострочные функции и могут быть использованы в следующих конструкциях:

- **Список выборки команды `SELECT`.**
- **Условие в предложениях `WHERE` и `HAVING`.**
- **Предложения `CONNECT BY`, `START WITH`, `ORDER BY` и `GROUP BY` в запросе.**
- **Предложение `VALUES` в команде `INSERT`.**
- **Предложение `SET` в команде `UPDATE`.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Вызов пользовательских функций

Пользовательская функция PL/SQL может быть вызвана из любого SQL-выражения, в котором допускается вызов встроенной однострочной функции.

Пример

```
SELECT employee_id, tax(salary)
FROM   employees
WHERE  tax(salary) > (SELECT MAX(tax(salary))
                      FROM employees
                      WHERE department_id = 30)
ORDER BY tax(salary) DESC;
```

EMPLOYEE_ID	TAX(SALARY)
100	1920
101	1360
102	1360
145	1120
146	1080
201	1040

...

10 rows selected.

Ограничения на вызов функций из SQL-выражений

- **Функция, вызываемая из SQL-выражения, должна:**
  - **храниться в базе данных**
  - **принимать параметры только в режиме `IN` с допустимыми типами данных SQL (но не с типами, присущими только PL/SQL)**
  - **возвращать тип данных, который допускается в SQL и не является специальным типом PL/SQL**
- **При вызове функций в командах SQL:**
  - **параметры должны задаваться позиционно**
  - **необходимо быть владельцем функции или иметь привилегию `EXECUTE`**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Ограничения на вызов функций из SQL-выражений

Пользовательская функция, написанная на PL/SQL, должна удовлетворять следующим требованиям, чтобы ее можно было вызвать из SQL-выражений.

- Функция должна храниться в базе данных.
- Функция может иметь только входные параметры с допустимые в SQL типами данных.
- Функция должны возвращать значение с типом данных SQL. Это не могут быть присущие только PL/SQL типы данных, например, `BOOLEAN`, `RECORD` или `TABLE`. Такие же ограничения накладываются на параметры функции.

Следующие ограничения накладываются на вызов функции из команды SQL:

- Параметры должны быть позиционными. Использование имен параметров не поддерживается.
- Необходимо быть владельцем функции или иметь привилегию `EXECUTE` для выполнение этой функции.

Остальные ограничения на пользовательские функции:

- Из нельзя вызывать из предложения по созданию ограничения `CHECK` в командах `CREATE` и `ALTER TABLE`.
- Их нельзя использовать для указания значения столбца по умолчанию.

Примечание. Только хранимые функции можно вызвать из команд SQL. Нельзя вызвать хранимые процедуры, кроме как из функций, удовлетворяющих выше приведенным ограничениям.

Контроль побочных эффектов при вызове функций из выражений SQL

- **Функции, вызываемые из команды `SELECT`, не могут содержать команды DML**
- **Функции, вызываемые из команды `UPDATE` или `DELETE`, выполняемой над таблицей `T`, не могут содержать запрос или DML-операции над той же самой таблицей `T`**
- **Функции, вызываемые из команд SQL, не могут завершать транзакции (то есть не могут выполнять операции `COMMIT` или `ROLLBACK`)**

Примечание: в функции недопустимы вызовы подпрограмм, нарушающих одно из этих ограничений.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Контроль побочных эффектов при вызове функций из выражений SQL

Чтобы выполнить команду SQL, вызывающую хранимую функцию, сервер Oracle должен быть уверен в том, что функция не имеет побочных эффектов. Побочные эффекты – это недопустимые изменения в таблицах базы данных.

Дополнительные ограничения действуют только, когда функция вызывается в выражении команды.

- Функция не может изменять данные в таблицах БД, если она вызывается из команды `SELECT` или параллельно выполняемых команд `UPDATE` и `DELETE`.
- Функция не может запрашивать и изменять данные в таблицах БД, если эта функция вызывается из команд `UPDATE` или `DELETE`, которые изменяют ту же самую таблицу.
- Функция, вызываемая из команд `SELECT`, `INSERT`, `UPDATE` или `DELETE`, не может выполнять прямо или косвенно SQL-команды управления транзакциями, такие как:
  - `COMMIT` или `ROLLBACK`;
  - команды управления в сеансе (например, `SET ROLE`);
  - команды управления системой (например, `ALTER SYSTEM`);
  - команды DDL (например, `CREATE`), поскольку перед ними производится автоматическая фиксация.

Ограничения на вызов функций из SQL: пример

```
CREATE OR REPLACE FUNCTION dml_call_sql(sal NUMBER)
   RETURN NUMBER IS
BEGIN
  INSERT INTO employees(employee_id, last_name,
                  email, hire_date, job_id, salary)
  VALUES(1, 'Frost', 'jfrost@company.com',
         SYSDATE, 'SA_MAN', sal);
  RETURN (sal + 100);
END;
```

```
UPDATE employees
  SET salary = dml_call_sql(2000)
WHERE employee_id = 170;
```

```
UPDATE employees SET salary = dml_call_sql(2000)
              *
ERROR at line 1:
ORA-04091: table PLSQL.EMPLOYEES is mutating,
trigger/function may not see it
ORA-06512: at "PLSQL.DML_CALL_SQL", line 4
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Ограничения на вызов функций из SQL: пример

Функция DML_CALL_SQL содержит команду DML, которая вставляет новую запись в таблицу EMPLOYEES и возвращает переданную величину оклада, увеличенную на 100. Эта функция вызывается из команды UPDATE, которая изменяет оклад служащего с номером 170 на величину, возвращаемую из функции. Команда UPDATE приводит к ошибке, которая указывает на то, что таблица в настоящий момент видоизменяется (mutating).

Рассмотрим следующий пример, в котором функция query_call_sql запрашивает столбец SALARY таблицы EMPLOYEE:

```
CREATE OR REPLACE FUNCTION query_call_sql(a NUMBER)
   RETURN NUMBER IS
   s NUMBER;
 BEGIN
   SELECT salary INTO s FROM employees
   WHERE employee_id = 170;
   RETURN (s + a);
 END;
```

Вызов этой функции из следующей команды UPDATE приведет к ошибке, схожей с приведенной на слайде.

```
UPDATE employees SET salary = query_call_sql(100)
WHERE employee_id = 170;
```

Удаление функций

Удаление хранимой функции:

- **Синтаксис:**

```
DROP FUNCTION имя_функции
```

Пример:

```
DROP FUNCTION get_sal;
```

- **Все привилегии, предоставленные на функцию, отбираются, когда функция удаляется.**
- **Использование синтаксиса `CREATE OR REPLACE` эквивалентно удалению и повторному созданию функции. Предоставленные привилегии остаются при использовании такого синтаксиса.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Удаление функций

Если хранимая функция больше не нужна, ее можно удалить в *i*SQL*Plus с помощью команды SQL `DROP FUNCTION`.

Сравнение `CREATE OR REPLACE` с `DROP` и `CREATE`

Фраза `REPLACE` в предложении `CREATE OR REPLACE` приводит к тому, что функция удаляется и повторно создается. При использовании синтаксиса `CREATE OR REPLACE` привилегии, предоставленные на этот объект другим пользователям, у них остаются. Напротив, при выполнении команды `DROP` для функции с последующим созданием этой функции все привилегии на эту функцию автоматически отбираются.

Просмотр информации о функциях в словаре данных

Сведения о функциях PL/SQL хранятся в следующих представлениях словаря данных:

- **USER_SOURCE содержит исходный код подпрограмм текущего пользователя; ALL_SOURCE дополнительно хранит подпрограммы других пользователей, предоставивших текущему пользователю привилегию их выполнения (EXECUTE).**

```
SELECT text
FROM   user_source
WHERE  type = 'FUNCTION'
ORDER BY line;
```

- **USER_OBJECTS содержит имена функций, принадлежащих текущему пользователю.**

```
SELECT object_name
FROM   user_objects
WHERE  object_type = 'FUNCTION';
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Просмотр информации о функциях в словаре данных

В таблицах словаря данных хранится исходный код подпрограмм PL/SQL, скомпилированных успешно или с ошибками. Для просмотра исходного кода используйте запросы (SELECT) к следующим представлениям (при условии TYPE ='FUNCTION'):

- USER_SOURCE (для вывода кода PL/SQL, принадлежащего текущему пользователю);
- ALL_SOURCE (для вывода кода PL/SQL, принадлежащего другим пользователям, предоставившим текущему пользователю привилегию EXECUTE для выполнения своей подпрограммы).

Первый запрос, приведенный на слайде, показывает, как вывести исходный код всех функций текущей схемы.

Второй запрос к представлению USER_OBJECTS возвращает имена всех функций текущего пользователя.

Сравнение процедур и функций

Процедура	Функция
Выполняется как команда PL/SQL	Вызывается как часть выражения
Нет предложения RETURN в заголовке	Заголовок должен содержать предложение RETURN
Может возвращать одно или несколько значений	Должна возвращать одно значение
Может содержать оператор RETURN	Должна содержать, по крайней мере, один оператор RETURN

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Чем отличаются процедуры и функции

Процедура создается для определения и сохранения последовательности действий, многократно исполняемых в будущем. Процедура может не иметь ни одного параметра или иметь некоторое количество параметров, передаваемых ей и возвращаемых из нее в вызывающую среду. Однако процедура необязательно должна возвращать значение. Для выполнения операций из процедуры можно вызывать функцию.

Примечание: процедуру, содержащую один параметр `OUT`, можно переписать как функцию, содержащую оператор `RETURN`.

Функция создается для вычисления значения, которое должно быть возвращено в вызывающую среду. Функция может не иметь ни одного параметра или иметь некоторое количество параметров, передаваемых из вызывающей среды. Функции должны возвращать только одно значение с помощью команды `RETURN`. Функции, используемые в командах SQL, не могут иметь параметры типа `OUT` и `IN OUT`. Функция с выходными параметрами может вызываться в процедуре PL/SQL или блоке, однако такую функцию нельзя использовать в командах SQL.

Итоги

- Создание функции по команде SQL `CREATE FUNCTION` для подсчета и возврата значения
- Вызов функции в выражении PL/SQL
- Использование хранимых функций PL/SQL в командах SQL
- Удаление функции из базы данных с помощью команды SQL `DROP FUNCTION`

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги

Функция – это именованный блок PL/SQL, который должен вернуть значение. Обычно функция используется для вычисления и возврата значения, а процедура –для выполнения действия.

Функция может быть создана и удалена.

Функция вызывается из выражения.

Обзор практического занятия 2

- **Создание хранимых функций**
  - **Для выполнения запроса к таблице базы данных и возврата отдельных значений**
  - **Для использования в команде SQL**
  - **Для вставки в таблицу базы данных новой строки со значениями, заданными в параметрах**
  - **Использование значений параметров по умолчанию**
- **Вызов хранимой функции из команды SQL**
- **Вызов хранимой функции из хранимой процедуры**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор практического занятия 2

Если при использовании *i*SQL*Plus возникнут ошибки компиляции, воспользуйтесь командой `SHOW ERRORS`.

Если вы исправляете какие-либо ошибки в *i*SQL*Plus, делайте это не в буфере, а в исходном скрипт-файле, после чего выполните новый вариант командного файла. При этом новая версия будет сохранена в словаре данных.

Практическое занятие 2

1. Создайте и вызовите функцию GET_JOB, возвращающую наименование должности.
   (а). Создайте функцию GET_JOB, возвращающую наименование должности.
   (б). Создайте хост-переменную TITLE с типом VARCHAR2 и VARCHAR2 и максимальной длиной в 35 символов . Вызовите функцию, передав ей идентификатор должности SA_REP. Чтобы просмотреть результат, выведите значение хост-переменной.

TITLE
Sales Representative

2. Создайте функцию GET_ANNUAL_COMP, возвращающую годовой заработок служащего. Функции передаются два параметра: месячный оклад служащего и размер комиссионных.
   (а). Создайте и вызовите функцию GET_ANNUAL_COMP, принимающую в качестве параметров месячный оклад и размер комиссионных. Одно или обо значений может быть NULL, однако функция должна вернуть определенное значение. Годовой заработок вычисляется по формуле:
   (salary*12) + (commission_pct*salary*12)
   (б). Используйте функцию в команде SELECT, осуществляющей выборку данных о служащих 30 отдела из таблицы EMPLOYEES..

EMPLOYEE_ID	LAST_NAME	Annual Compensation
114	Raphaely	132000
115	Khoo	37200
116	Baida	34800
117	Tobias	33600
118	Himuro	31200
119	Colmenares	30000

6 rows selected.

3. Созд ицу
   EMPI проверки
   того,
   DEPARTMENTS.
   (а). Создайте функцию VALID_DEPTID для проверки заданного номера отдела. Функция должна возвращать логическое значение TRUE, если отдел существует.

Практическое занятие 2

(б). Создайте процедуру `ADD_EMPLOYEE` для вставки нового служащего в таблицу `EMPLOYEES`. Запись о новом служащем должна быть добавлена в таблицу `EMPLOYEES` только в случае, если функция возвращает значение `TRUE`. Если же функция возвращает значение `FALSE`, процедура должна выдавать пользователю соответствующее предупреждение. Задайте следующие параметры: `first_name`, `last_name`, `email`, `job`, `mgr`, `sal`, `comm` и `deptid`. По умолчанию должность (`job`) – `SA_REP`, номер менеджера (`mgr`) – 145, оклад (`sal`) – 1000, размер комиссионных (`comm`) – 0 и номер отдела (`deptid`) – 30. Для номера служащего используйте последовательность `EMPLOYEES_SEQ`. Задайте дату найма `hire_date` как `TRUNC(SYSDATE)`.

(в). Для проверки процедуры `ADD_EMPLOYEE` добавьте нового служащего `Jane Harris` в отдел 15. Для остальных параметров используйте значения по умолчанию. Каков результат?

(г). Добавьте служащего `Joe Harris` в отдел 80. Для остальных параметров используйте значения по умолчанию. Каков результат?